

Assignment 2: Coding Basics

Juanying Huangfu

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.
7. Initial here to acknowledge that you did not use AI at all in completing this assignment: *Juanying Huangfu*

Basics, Part 1

1. Use R’s `seq()` function to create a sequence of numbers from 100 to 333, increasing by threes. Assign this sequence a variable name.
2. Compute the *mean* of this sequence, assigning this values its own variable name.
3. Compute the *standard deviation* (`sd()`) of this sequence, assigning this values its own variable name.
4. Display the the mean minus the standard deviation as well as the mean plus the standard deviation.
5. Insert comments in your code to describe what you are doing.

```
#1. Create a sequence from 100 to 333, increasing by 3
num_seq <- seq(from = 100, to = 333, by = 3)

#2. Compute the mean of the sequence
seq_mean <- mean(num_seq)

#3. Compute the standard deviation of the sequence
seq_sd <- sd(num_seq)

#4. Display the mean minus the standard deviation
seq_mean - seq_sd
```

```
## [1] 147.5184
```

```
# Display the mean plus the standard deviation  
seq_mean + seq_sd
```

```
## [1] 283.4816
```

Basics, Part 2

6. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
7. Label each vector with a comment on what type of vector it is.
8. Combine each of the vectors into a data frame. Assign the data frame an informative name.
9. Label the columns of your data frame with informative titles.

```
#6. Create a character vector of student names  
student_names <- c("Anna", "Bob", "Cathy", "David")  
  
# Create a numeric vector of test scores  
test_scores <- c(35, 45, 85, 95)  
  
# Create a logical vector indicating scholarship status  
on_scholarship <- c(FALSE, FALSE, TRUE, TRUE)  
  
#7. Comments on vector types:  
# student_names is a character vector  
# test_scores is a numeric vector  
# on_scholarship is a logical vector  
  
#8. Combine the vectors into a data frame  
student_data <- data.frame(  
  Name = student_names,  
  Score = test_scores,  
  Scholarship = on_scholarship  
)  
  
#9. The column names are already labeled as:  
# Name, Score, and Scholarship  
# Display the data frame  
student_data
```

```
##      Name Score Scholarship  
## 1  Anna    35      FALSE  
## 2   Bob    45      FALSE  
## 3 Cathy    85       TRUE  
## 4 David    95       TRUE
```

10. QUESTION: How is this data frame different from a matrix?

Answer: A data frame can hold different types of data in each column (for example, characters, numbers, and logical values), while a matrix can only hold one data type. Also, data frames are designed to store table-like data with meaningful column names, whereas matrices are mainly used for mathematical operations.

Basics, Part 3

11. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, it returns the word “Pass”; otherwise it returns the word “Fail”.
12. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
13. Run both functions using the value 54 as the input
14. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#11. Create a function using if...else

```
pass_fail_if <- function(x) {  
  if (x > 50) {  
    return("Pass")  
  } else {  
    return("Fail")  
  }  
}
```

#12. Create a function using ifelse()

```
pass_fail_ifelse <- function(x) {  
  ifelse(x > 50, "Pass", "Fail")  
}
```

#13a. Run the first function with the value 54

```
pass_fail_if(54)
```

```
## [1] "Pass"
```

#13b. Run the second function with the value 54

```
pass_fail_ifelse(54)
```

```
## [1] "Pass"
```

#14a. Run the first function with the vector of test scores

```
#pass_fail_if(test_scores)
```

#14b. Run the second function with the vector of test scores

```
pass_fail_ifelse(test_scores)
```

```
## [1] "Fail" "Fail" "Pass" "Pass"
```

15. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”; it’s ok here if an AI response is presented in the search response.)

Answer: `ifelse()` worked, but `if ... else` did not. This is because `ifelse()` is vectorized, meaning it can apply a condition to each element of a vector. In contrast, `if ... else` is designed to work with a single logical value, not a vector, so it fails when given multiple values at once.

NOTE Before knitting, you’ll need to comment out the call to the function in Q14 that does not work. (A document can’t knit if the code it contains causes an error!)